Things About MoE

HJ

November 3, 2026

1 Introduction to MoE

1.1 Arch

A Mixture-of-Experts layer replaces the single dense FFN in a transformer block with:

- N **Experts**: N smaller feed-forward networks, each an FFN that substitutes the single dense FFN.
- 1 **Router (gate)**: a small linear layer W_g of shape hidden $\times N$ that scores every expert for each token and selects the top-k.

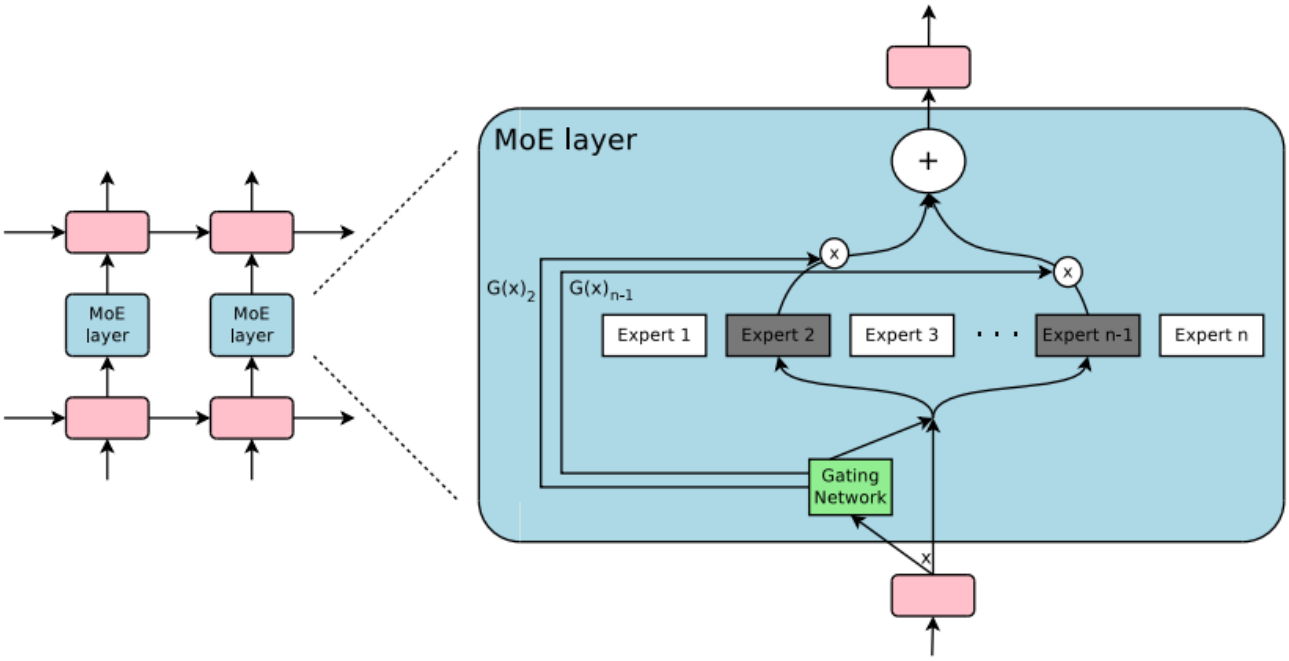


Figure 1: MoE layer: a router selects top-k experts for each token.

The key trade-off is that capacity depends on total parameters while inference speed depends on active parameters: only top-k experts run per token, so per-token FLOPs stay comparable to a dense FFN while total parameters grow $\sim N\times$. For example, DeepSeek-V4-Flash uses 256 routed experts with top-6 active, hidden size 4096, and per-expert intermediate 2048.

1.2 Noise & Load Balance

The output is a weighted sum of the selected experts:

$$y = \sum_{i=1}^{N} G(x)_i \, E_i(x)$$

where the gating $G(x)$ is a softmax over the top-k raw scores:

$$G(x) = \mathrm{Softmax}\big(\mathrm{KeepTopK}(H(x), k)\big)$$

and the raw scores $H(x)$ add a tunable Gaussian noise term used for load balancing during training:

$$H(x)_i = (x\,W_g)_i + \mathrm{StandardNormal}() \cdot \mathrm{Softplus}\big((x\,W_{noise})_i\big)$$

Here KeepTopK(v, k) keeps the top-k entries of v and sets the rest to $-\infty$, so the softmax concentrates all mass on the selected experts. The weights $g_i = G(x)_i$ are dynamic activations computed per token, while only W_g and W_{noise} are static parameters. Softmax is one gating choice; DeepSeek instead uses sigmoid scores with re-normalization.

2 GShard & Expert Parallelism

Obviously, since tokens are dispatched to experts, the expert demension has naturally parallelism. GShard scales MoE with expert parallelism plus all-to-all dispatch/combine, where all-to-all is a whole-cluster operation (the entire cluster is one group) rather than a per-node exchange. The round trip per MoE layer is:

1. **Sharding**: the input $[B, L, E]$ is sharded along the token dimension; the layout starts uniform/balanced.
2. **Router + All2All dispatch**: each device gates its local tokens, then dispatches each token to the device holding its assigned experts; the distribution becomes unbalanced (skewed).
3. **Expert + All2All combine**: each device runs its own experts, then combine sends results back and performs the weighted sum; the layout returns to the original balanced one.

About the Notations: Usually $[B, L, E]$ are used in training, but only $[M, E]$ in trainning, sometimes B can be seen as number of request.

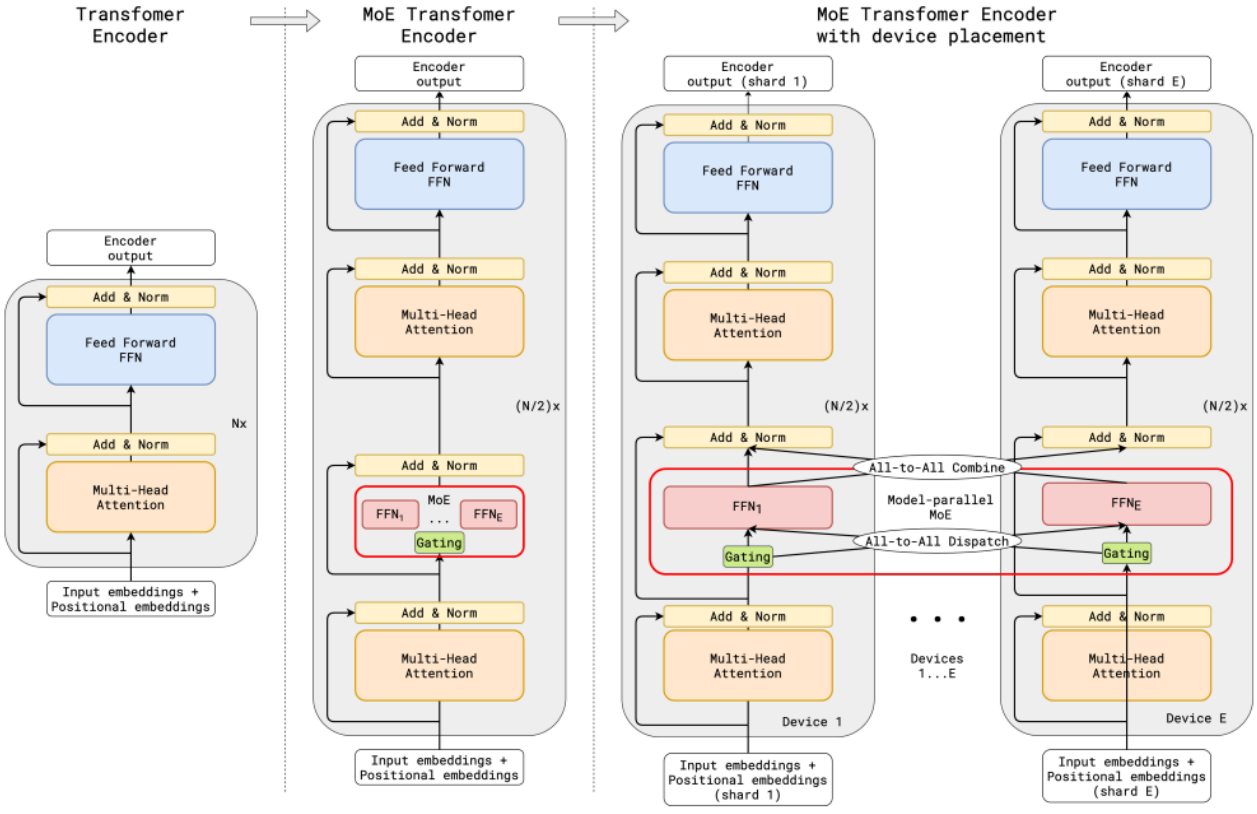


Figure 2: GShard: token sharding, dispatch (all-to-all), expert compute, combine (all-to-all).

3 Implementation

TODO: Learn SGLang and weight loading

References

[1] O. Sanseviero et al., *Mixture of Experts Explained*, Hugging Face Blog, 2023, huggingface.co/blog/moe.

[2] A. R. Gosthipaty et al., *Mixture of Experts (MoEs) in Transformers*, Hugging Face Blog, 2026, huggingface.co/blog/moe-transformers.

[3] D. Lepikhin et al., *GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding*, 2020, arXiv:2006.16668.